

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA
FACULTAD DE INGENIERIA DE PRODUCCION Y SERVICIOS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



PROYECTO FINAL

Documentación técnica: Admin Linux

Curso : Programación de Sistemas

Docente : Mg. Norman Patrick Harvey Arce

Elaborado por : Choque Sánchez, Alejandra Camila
Mamani Céspedes, Jhonatan Benjamin
Quispe Paucar, Josue Claudio
Carrillo Villalta, Gustavo Alonso

Arequipa - Perú
26 julio 2026

1. Introducción

El presente documento constituye la documentación técnica del Proyecto Final del curso de Programación de Sistemas, correspondiente al desarrollo de una herramienta de administración de sistemas para entornos Linux, denominada ADMIN LINUX. El sistema se implementa en lenguaje C sobre el estándar C11, siguiendo un enfoque modular que integra funcionalidades de monitoreo de procesos, gestión de archivos, ejecución de comandos del sistema operativo, respaldo de información, análisis de scripts Bash y administración de una cola de descargas.

El objetivo de este documento es describir la arquitectura del software, las decisiones de diseño adoptadas, la organización del código fuente, las herramientas de compilación empleadas y el estado de implementación de cada módulo, de manera que sirva como referencia técnica para la comprensión, mantenimiento y extensión del sistema.

2. Objetivo del proyecto

Desarrollar una herramienta de administración para Linux que integre monitoreo de procesos, gestión de archivos, respaldo automático, ejecución de comandos y análisis de scripts Bash, utilizando programación en lenguaje C y las facilidades que ofrece el sistema operativo GNU/Linux a nivel de llamadas al sistema (system calls) y utilidades estándar.

2.1. Objetivos específicos

- Diseñar una arquitectura modular que permita el desarrollo independiente y colaborativo de cada componente del sistema
- Implementar un administrador de procesos capaz de listar, buscar, monitorear y controlar procesos del sistema operativo
- Implementar un shell de archivos que permita operaciones básicas de gestión de archivos y directorios
- Proveer una interfaz para la ejecución de comandos Linux con captura de salida estándar y de errores
- Implementar un sistema de respaldos incrementales con posibilidad de restauración de versiones
- Desarrollar un analizador de scripts Bash capaz de detectar variables y estructuras de control (ciclos)
- Implementar una cola de descargas con seguimiento de progreso y eventos

3. Alcance

El alcance del proyecto comprende el diseño, implementación, documentación y presentación de una aplicación de consola (CLI) ejecutable en distribuciones Linux basadas en Ubuntu. La aplicación se estructura en seis módulos funcionales

controlados por un menú principal, además de un módulo de utilidades transversales. El desarrollo se realiza bajo control de versiones con Git, empleando un flujo de trabajo colaborativo mediante ramas.

4. Arquitectura del sistema

ADMIN LINUX sigue una arquitectura modular con un menú controlador, que como un módulo central orquesta la invocación de submódulos independientes, cada uno responsable de una funcionalidad específica del sistema. Este patrón facilita el trabajo colaborativo en equipo, dado que cada integrante puede desarrollar y probar su módulo de forma aislada, respetando una interfaz pública común expuesta a través de archivos de cabecera.

4.1. Diagrama de módulos

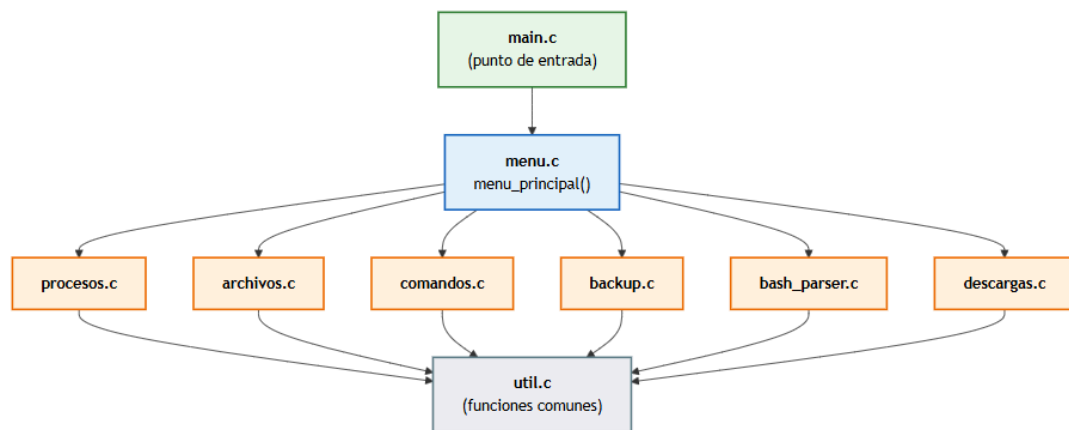


Figura 1. Diagrama de módulos ADMIN LINUX

4.2. Principios de diseño aplicados

- **Modularidad**: cada funcionalidad (procesos, archivos, comandos, respaldos, bash, descargas) se implementa en un archivo .c con su respectivo archivo de cabecera .h, exponiendo una única función pública de entrada (por ejemplo, menu_procesos()).
- **Bajo acoplamiento**: los módulos no dependen entre sí directamente; toda comunicación con la interfaz de usuario pasa por el módulo menu y las utilidades comunes del módulo util.
- **Punto de entrada único**: main.c se limita a invocar menu_principal(), delegando toda la lógica de navegación al módulo de menú.
- **Reutilización de código**: funciones transversales como limpiar_pantalla() y pausar() se centralizan en util.c/util.h para ser utilizadas por todos los módulos.
- **Separación de interfaz e implementación**: los archivos .h definen contratos (prototipos) mientras que los archivos .c contienen la implementación, permitiendo compilación independiente por unidades de traducción.

4.3. Estructura de directorios



Figura 2. Estructura de directorios ADMIN LINUX

Los directorios backups, downloads, historial, logs, scripts y docs se encuentran reservados en el repositorio como puntos de extensión para el almacenamiento de datos generados en tiempo de ejecución por los módulos correspondientes, conforme su implementación.

5. Tecnologías y herramientas

Componente	Herramienta / Versión	Propósito
Lenguaje	C (estándar C11)	Implementación del sistema, acceso a APIs del sistema operativo
Compilador	GCC (GNU Compiler Collection)	Compilación del código fuente C a código objeto y ejecutable

Automatización	GNU Make	Gestión de la compilación mediante Makefile
Control de versiones	Git	Versionado del código y trabajo colaborativo en ramas
Sistema operativo	Ubuntu 22.04 o superior	Plataforma objetivo de ejecución
Depuración	GDB	Depuración de errores en tiempo de ejecución
Utilidades del sistema	procps, psmisc, rsync, tree, curl, wget	Soporte a los módulos de procesos, respaldos y descargas

El proyecto no utiliza librerías externas de terceros: la implementación se apoya en la biblioteca estándar de C (stdio.h, stdlib.h) y en llamadas al sistema y comandos propios de GNU/Linux, en concordancia con la Programación de Sistemas orientados a la interacción directa entre el programa en C y el sistema operativo.

6. Compilación y ejecución

6.1. Flags de compilación

El Makefile del proyecto define las siguientes banderas de compilación para GCC:

```
CC = gcc
CFLAGS = -Wall -Wextra -std=c11 -Iinclude
```

- -Wall y -Wextra: habilitan advertencias exhaustivas del compilador, favoreciendo la detección temprana de errores.
- -std=c11: fija el estándar del lenguaje en C11.
- -Iinclude: agrega el directorio include/ a la ruta de búsqueda de cabeceras.

6.2. Proceso de compilación

La compilación se realiza de forma automatizada mediante el Makefile, el cual detecta dinámicamente todos los archivos fuente del directorio y genera un archivo objeto por cada uno de ellos antes de enlazarlos en el ejecutable final `admin_linux`.

```
$ make          # Compila el proyecto completo
$ make run      # Compila (si es necesario) y ejecuta el programa
$ make clean    # Elimina archivos objeto (.o) y el ejecutable
```

6.3. Instalación de dependencias

El script `requirements.sh` automatiza la instalación de las herramientas necesarias en distribuciones basadas en Debian/Ubuntu mediante el gestor de paquetes APT:

```
$ chmod +x requirements.sh
$ ./requirements.sh

Paquetes instalados:
build-essential, gcc, make, gdb, git, tree,
procps, psmisc, rsync, curl, wget, zip, unzip
```

7. Descripción de módulos

A continuación se describe cada módulo del sistema, indicando su responsabilidad, las funciones especificadas para su alcance final y su estado actual de implementación dentro del ciclo de desarrollo del proyecto.

7.1. Módulo menu (menu.c / menu.h)

Responsable de presentar el menú principal del sistema y dirigir el flujo de ejecución hacia el submódulo correspondiente según la opción seleccionada por el usuario. Implementa un ciclo do-while controlado por la variable opción, que se mantiene activo hasta que el usuario selecciona la opción de salida (0).

7.2. Módulo procesos (procesos.c / procesos.h) : Administrador de tareas

Módulo destinado a la gestión de procesos del sistema operativo, apoyándose en la lectura del pseudo-sistema de archivos /proc y en llamadas al sistema estándar de POSIX.

Funcionalidad especificada
Listar procesos activos del sistema
Buscar procesos por nombre
Mostrar consumo de CPU y memoria por proceso
Finalizar procesos (señal SIGTERM/SIGKILL)
Suspender y reanudar procesos (señales SIGSTOP/SIGCONT)
Visualizar árbol de procesos (relación padre-hijo, PID/PPID)

7.3. Módulo archivos (archivos.c / archivos.h) : Shell de Archivos

Módulo orientado a la gestión del sistema de archivos, mediante funciones estándar de C (dirent.h, sys/stat.h, unistd.h) y utilidades del sistema.

Funcionalidad especificada
Listar archivos y directorios
Copiar y mover archivos
Eliminar archivos
Búsqueda de archivos por nombre o patrón

Estadísticas de archivos (tamaño, permisos, fechas)
Listar archivos y directorios

7.4. Módulo comandos Linux (comandos.c / comandos.h)

Módulo encargado de la ejecución de comandos arbitrarios del sistema operativo desde la aplicación, capturando de forma diferenciada la salida estándar (stdout) y la salida de error (stderr), típicamente mediante popen() o combinaciones de fork()/exec()/pipe().

Funcionalidad especificada
Ejecutar comandos del sistema ingresados por el usuario
Mostrar la salida estándar del comando
Mostrar la salida de error del comando
Registrar historial de comandos ejecutados

7.5. Módulo de respaldos backup (backup.c / backup.h)

Módulo destinado a la generación de respaldos incrementales de archivos y directorios, con soporte de restauración de versiones anteriores. Se apoya conceptualmente en utilidades como rsync y en el manejo de metadatos de archivos (marcas de tiempo) para determinar cambios.

Funcionalidad especificada
Generar respaldos incrementales
Restaurar versiones previas de un respaldo
Almacenar los respaldos en el directorio backups/

7.6. Módulo analizador de scripts Bash (bash_parser.c / bash_parser.h)

Módulo orientado al análisis estático de scripts Bash (.sh), con el propósito de identificar declaraciones de variables y estructuras de control repetitivas (ciclos for, while, until), útil como herramienta de apoyo a la comprensión y depuración de scripts del sistema.

Funcionalidad especificada
Analizar el contenido de un script Bash
Detectar variables declaradas en el script
Detectar ciclos (for, while, until)

7.7. Módulo analizador de scripts Bash (bash_parser.c / bash_parser.h)

Módulo responsable de administrar una cola de descargas de archivos, reportando el progreso y los eventos relevantes de cada tarea ya sea de inicio, avance,

finalización y error, típicamente en integración con herramientas como wget o curl.

Funcionalidad especificada
Administrar una cola de descargas
Mostrar el progreso de cada descarga
Almacenar los archivos descargados en downloads/
Registrar eventos de la descarga (inicio, error, fin)

8. Consideraciones técnicas y buenas prácticas

- El proyecto utiliza exclusivamente la biblioteca estándar de C y utilidades nativas de GNU/Linux, en línea con el enfoque del curso sobre programación a nivel de sistema.
- Las advertencias del compilador deben mantenerse en cero antes de cada entrega, a fin de garantizar un código robusto y libre de comportamientos indefinidos.
- Las funciones de manejo de pantalla y pausa deben reutilizarse desde util.h en lugar de duplicarse en cada módulo.

9. Pruebas de uso y ejecución

Módulo de backups:

```
asusjosue@JOSUE: /mnt/c/Us  X  +  v
=====
TRABAJOS DE RESPALDO
=====

- josue (version actual: 2)

Presione ENTER para continuar...|
```

```
asusjosue@JOSUE: /mnt/c/Us  X  +  v
=====
VERSIONES DE UN TRABAJO
=====

Nombre del trabajo: josue

Versiones disponibles para 'josue':

Version 1:
version=1
fecha=2026-07-26 23:07:01
origen=/home/asusjosue/prueba_origen
agregados=3
eliminados=0

Version 2:
version=2
fecha=2026-07-26 23:08:03
origen=/home/asusjosue/prueba_origen
agregados=2
eliminados=1

Presione ENTER para continuar...|
```



```
=====
CREAR / ACTUALIZAR RESPALDO
=====

Nombre del trabajo de respaldo: josue
Trabajo existente. Se usara el origen ya registrado:
/home/asusjosue/prueba_origen

Respaldo completado: version 3
Archivos nuevos o modificados copiados: 0
Archivos eliminados detectados: 0

Presione ENTER para continuar...|
```

```
=====
RESTAURAR VERSION
=====

Nombre del trabajo: josue
Este trabajo tiene 3 version(es) disponibles.
Version a restaurar: 2
Directorio de destino para restaurar: /home/asusjosue/prueba_origen

Restauracion completada (version objetivo: 2).
Archivos restaurados: 3 de 3

Presione ENTER para continuar...|
```

```
=====
VERSIONES DE UN TRABAJO
=====

Nombre del trabajo: josue

Versiones disponibles para 'josue':

Version 1:
  version=1
  fecha=2026-07-26 23:07:01
  origen=/home/asusjosue/prueba_origen
  agregados=3
  eliminados=0

Version 2:
  version=2
  fecha=2026-07-26 23:08:03
  origen=/home/asusjosue/prueba_origen
  agregados=2
  eliminados=1

Version 3:
  version=3
  fecha=2026-07-26 23:23:02
  origen=/home/asusjosue/prueba_origen
  agregados=0
  eliminados=0
```

Módulo de analizador Bash:

```
asusjosue@JOSUE: /mnt/c/Us  x  +  v  -  □  x

=====
ANALIZADOR BASH
=====

Ruta del script (.sh) a analizar: /home/asusjosue/prueba.sh

===== RESUMEN DEL ANALISIS =====
Archivo analizado: /home/asusjosue/prueba.sh

Lineas totales: 14
Lineas de codigo: 13
Lineas de comentario: 1
Lineas en blanco: 0

Bucles detectados:
  for: 1
  while: 1
  until: 0

Estructuras condicionales:
  if/elif: 1
  case: 0

Funciones definidas (1):
  - saludar

Variables declaradas (2):
  - NOMBRE
  - i

Resumen general:
El script posee 14 lineas en total (13 de codigo, 1 de comentarios y 0 en blanco).
Se identificaron 1 bucle(s) for, 1 bucle(s) while y 0 bucle(s) until, ademas de 1 funcion(es) y 2 variable(s) declaradas.

=====

Reporte guardado en: logs/prueba.sh_reporte.txt

Presione ENTER para continuar...|
```

```
EXPLORADOR  ...  prueba.sh_reporte.txt x
> EDITORES ABIERTOS
  > ADMIN LINUX
    > backups\josue
    > historial
    # comandos.log
    > include
      C archivos.h
      C backup.h
      C bash_parser.h
      C comandos.h
      C descargash.h
      C menu.h
      C procesos.h
      C util.h
    > logs
      # prueba.sh_reporte.txt
    > src
      C archivos.c
      # archivos.o
      C backup.c
      # backup.o
      C bash_parser.c
      # bash_parser.o
      C comandos.c
      # comandos.o
      C descargas.c

logs > prueba.sh_reporte.txt
1
2  ===== RESUMEN DEL ANALISIS =====
3  Archivo analizado: /home/asusjosue/prueba.sh
4
5  Lineas totales: 14
6  Lineas de codigo: 13
7  Lineas de comentario: 1
8  Lineas en blanco: 0
9
10 Bucles detectados:
11   for: 1
12   while: 1
13   until: 0
14
15 Estructuras condicionales:
16   if/elif: 1
17   case: 0
18
19 Funciones definidas (1):
20   - saludar
21
22 Variables declaradas (2):
23   - NOMBRE
24   - i
25
26 Resumen general:
27 El script posee 14 lineas en total (13 de codigo, 1 de comentarios y 0 en blanco).
28 Se identificaron 1 bucle(s) for, 1 bucle(s) while y 0 bucle(s) until, ademas de 1 funcion(es) y 2 variable(s) declaradas.
29 =====
30
```

10. Conclusiones

La arquitectura modular adoptada por ADMIN LINUX permite un desarrollo colaborativo ordenado, en el cual cada funcionalidad del sistema de administración Linux puede evolucionar de forma independiente sin comprometer la estabilidad del sistema de menús principal. La base de compilación mediante Makefile y la organización de directorios reservados para datos en tiempo de ejecución sientan las condiciones necesarias para completar la implementación funcional de cada módulo

en las siguientes etapas del proyecto, conforme a lo establecido en la especificación entregada por el curso.

11. Referencias

Kerrisk, M. (2010). *The Linux programming interface: A Linux and UNIX system programming handbook*. No Starch Press.

Love, R. (2013). *Linux system programming: Talking directly to the kernel and C library* (2nd ed.). O'Reilly Media.

Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language* (2nd ed.). Prentice Hall.

Linux Programmer's Manual. (2021). *popen(3) — pipe stream to or from a process*. <https://man7.org/linux/man-pages/man3/popen.3.html>

The IEEE and The Open Group. (2018). *POSIX.1-2017: Shell and utilities, issue 7*. <https://pubs.opengroup.org/onlinepubs/9699919799/utilities/contents.html>

Robbins, A. (2004). *Linux programming by example: The Open Source philosophy*. Prentice Hall.

Stallman, R., McGrath, R., & Smith, M. (2020). *GNU Make: A program for directing compilation, edition 4.3*. Free Software Foundation. <https://www.gnu.org/software/make/manual/make.html>

12. Anexos

Anexo 01: Repositorio grupal de control de versiones

El repositorio contiene la totalidad del código fuente estructurado (directorios src/ e include/), herramientas de automatización de compilación (Makefile), scripts de dependencias (requirements.sh) e historial completo de confirmaciones (commits) que validan el aporte técnico y la autoría del equipo de desarrollo.

https://github.com/JosueClaudioQP/ProyectoFinal_Psis

Anexo 02: Instrucciones de clonación y despliegue local

Secuencia de comandos recomendada:

Clonar el repositorio de forma local:

```
git clone https://github.com/JosueClaudioOP/ProyectoFinal\_Psis.git
```

Acceder al directorio del proyecto:

```
cd ProyectoFinal_Psis
```

Asignar permisos y aprovisionar dependencias:

```
chmod +x requirements.sh && ./requirements.sh
```

Compilar y ejecutar la aplicación mediante GNU Make:

```
make run
```